

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Nguyễn Vũ Thái¹ Bùi Văn Dũng² Đỗ Trí Nhựt² Nguyễn Văn Dũ³
SOICT 2026

¹Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam

²Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam

³Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Nguyễn Vũ Thái¹ Bùi Văn Dũng² Đỗ Trí Nhựt² Nguyễn Văn Dũ³
SOICT 2026

¹Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam

²Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam

³Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

Câu hỏi của bài: có hai bo mạch biên thì nên chia việc phát hiện xâm nhập giữa chúng thế nào, và bo thứ hai thực sự đem lại bao nhiêu.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

I. VẤN ĐỀ VÀ ĐÓNG GÓP

I. VẤN ĐỀ VÀ ĐÓNG GÓP

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Phát hiện xâm nhập ngay tại thiết bị biên: còn ba khoảng trống

Mô hình phát hiện xâm nhập này đã chạy được trên bo mạch nhỏ, nhưng nếu một bo phải xét **mọi** luồng thì nó quá tải rất nhanh.

Nghiên cứu trước còn thiếu

- 1. Chỉ đo trên **một thiết bị**, không chia việc cho nhiều bo.
- 2. Hiếm khi báo **cùng lúc** tốc độ, độ trễ và năng lượng.
- 3. Điểm số offline lấy từ quy trình **rò rỉ dữ liệu**.

Câu hỏi

Có hai bo Jetson thì nên *đặt* việc phát hiện lên chúng thế nào?

Cách làm

Tách làm hai: **cổng lọc** rẻ trên bo #1, **bộ phân loại** nặng trên bo #2, nối bằng Kafka.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Khoảng trống thứ ba là lý do nhóm làm bài FAIR về quy trình đánh giá chống rò rỉ.

Phát hiện xâm nhập ngay tại thiết bị biên: còn ba khoảng trống

Mô hình phát hiện xâm nhập này đã chạy được trên bo mạch nhỏ, nhưng nếu một bo phải xét **mọi** luồng thì nó quá tải rất nhanh.

Nghiên cứu trước còn thiếu

1. Chỉ đo trên **một thiết bị**, không chia việc cho nhiều bo.

2. Hiếm khi báo **cùng lúc** tốc độ, độ trễ và năng lượng.

3. Điểm số offline lấy từ quy trình **rò rỉ dữ liệu**.

Câu hỏi

Có hai bo Jetson thì nên đặt việc phát hiện lên chúng thế nào?

Cách làm

Tách làm hai: **cổng lọc** rẻ trên bo #1, **bộ phân loại** nặng trên bo #2, nối bằng Kafka.

- 1. Hệ thống phát hiện xâm nhập **hai tầng, phân tán**: cổng lọc và bộ phân loại Spark đặt trên hai bo khác nhau.
- 2. **Ba cách triển khai** trên cụm hai Jetson: tách tầng, nhân bản ngang, cụm Spark.
- 3. **Đo đầu cuối**: tốc độ, độ trễ, CPU/RAM từng bo và năng lượng mỗi phán quyết.
- 4. Mã nguồn, cấu hình và kịch bản đo **công khai**, chạy lại được.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

I. VẤN ĐỀ VÀ ĐÓNG GÓP

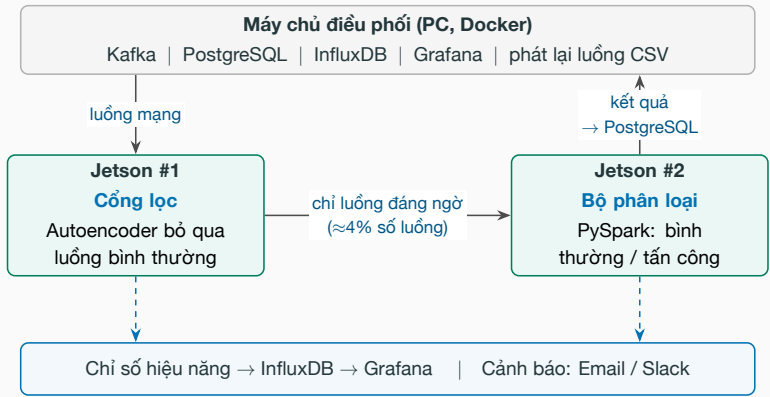
Phán quyết (verdict) là quyết định cuối cùng bình thường hay tấn công cho một luồng.

Các đóng góp

- 1. Hệ thống phát hiện xâm nhập **hai tầng, phân tán**: cổng lọc và bộ phân loại Spark đặt trên hai bo khác nhau.
- 2. **Ba cách triển khai** trên cụm hai Jetson: tách tầng, nhân bản ngang, cụm Spark.
- 3. **Đo đầu cuối**: tốc độ, độ trễ, CPU/RAM từng bo và năng lượng mỗi phán quyết.
- 4. Mã nguồn, cấu hình và kịch bản đo **công khai**, chạy lại được.

II. HỆ THỐNG VÀ CÁCH ĐO

Kiến trúc: tách công lọc và bộ phân loại ra hai bo

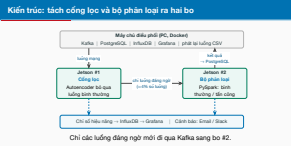


Chỉ các luồng đáng ngờ mới đi qua Kafka sang bo #2.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark
II. HỆ THỐNG VÀ CÁCH ĐO

Máy chủ chạy hạ tầng. Bo một lọc, bo hai phân loại. Mọi chỉ số đi vào Grafana theo thời gian thực.



Hai tầng làm việc thế nào

Tầng 1 — cổng lọc (autoencoder)

- Học cách *tái tạo* lưu lượng bình thường.
- Luồng tái tạo tốt (sai số nhỏ) → coi là bình thường, **dừng luôn**.
- Luồng tái tạo kém → đáng ngờ, chuyển sang tầng 2.
- Ngưỡng sai số chọn *trước* trên 10% dữ liệu bình thường, không nhìn tập kiểm tra.

Tầng 2 — bộ phân loại (PySpark)

- Random Forest trên **29 đặc trưng** được SHAP chọn, không có cổng đích.
- Chỉ phải xét số ít luồng mà cổng chuyển sang.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

II. HỆ THỐNG VÀ CÁCH ĐO

Sai số tái tạo là MSE. Ngưỡng là phân vị 0,995 của MSE trên tập kiểm định gồm 10% luồng bình thường của tập huấn luyện (jetson/model/anomaly_threshold.json). Bộ 29 đặc trưng (jetson/model/feature_columns.json) lấy từ một bảng xếp hạng SHAP cũ đã bỏ destination_port; nó chỉ trùng 15/30 với SHAP Top-30 hiện tại của bài FAIR, xem jetson/config.py.

Hai tầng làm việc thế nào	
Tầng 1 — cổng lọc (autoencoder) ■ Học cách tái tạo lưu lượng bình thường. ■ Luồng tái tạo tốt (sai số nhỏ) → coi là bình thường, dừng luôn. ■ Luồng tái tạo kém → đáng ngờ, chuyển sang tầng 2. ■ Ngưỡng sai số chọn trước trên 10% dữ liệu bình thường, không nhìn tập kiểm tra.	Tầng 2 — bộ phân loại (PySpark) ■ Random Forest trên 29 đặc trưng được SHAP chọn, không có cổng đích. ■ Chỉ phải xét số ít luồng mà cổng chuyển sang.

Vì sao bộ phân loại là Random Forest trên Spark

- **Random Forest có sẵn trong Spark ML**, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- **Ít đặc trưng** (29 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- **Cùng một bộ công cụ** (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 5.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

II. HỆ THỐNG VÀ CÁCH ĐO

Bộ 29 đặc trưng của mô hình đang chạy lấy từ một bảng xếp hạng SHAP cũ, không phải SHAP Top-30 hiện tại của bài FAIR (chỉ trùng 15/30). Vì vậy slide này không trích F1 0,9955 của bài FAIR cho mô hình triển khai. Các số đo tốc độ, độ trễ, năng lượng vẫn là của đúng mô hình đang chạy.

- **Random Forest có sẵn trong Spark ML**, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- **Ít đặc trưng** (29 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- **Cùng một bộ công cụ** (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 5.

Ba cách triển khai, cùng một khối lượng việc

Một bo làm hết mọi việc — mốc so sánh.

A: Tách tầng cổng trên #1, phân loại trên #2.

B: Nhân bản mỗi bo làm đủ cả hai tầng, chia nhau luồng qua Kafka.

C: Cụm Spark PC làm master, hai bo làm worker.

Tốc độ tính bằng **số phán quyết mỗi giây** — một luồng được quyết ở cổng hoặc ở bộ phân loại, không đếm hai lần.

Thiết lập đo

2× Jetson Orin Nano Super: CPU 6 nhân, 8 GB RAM, GPU ≈67 TOPS
PySpark 3.4.1, Kafka 3.5, mạng Wi-Fi
Phát lại CICIDS2017, 100 luồng/giây
Mỗi cấu hình lặp ≥3 lần, bỏ giai đoạn khởi động
Năng lượng đo bằng tegrastats

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

II. HỆ THỐNG VÀ CÁCH ĐO

Độ trễ p95 tính theo từng bo trên dữ liệu thô của từng luồng, và lấy bo tệ nhất làm con số của cả cụm. Bỏ giai đoạn khởi động để JVM chưa nóng máy không làm phồng độ trễ.

Ba cách triển khai, cùng một khối lượng việc	
Một bo làm hết mọi việc — mốc so sánh. A: Tách tầng cổng trên #1, phân loại trên #2. B: Nhân bản mỗi bo làm đủ cả hai tầng, chia nhau luồng qua Kafka. C: Cụm Spark PC làm master, hai bo làm worker. Tốc độ tính bằng số phán quyết mỗi giây — một luồng được quyết ở cổng hoặc ở bộ phân loại, không đếm hai lần.	Thiết lập đo 2× Jetson Orin Nano Super: CPU 6 nhân, 8 GB RAM, GPU ≈67 TOPS PySpark 3.4.1, Kafka 3.5, mạng Wi-Fi Phát lại CICIDS2017, 100 luồng/giây Mỗi cấu hình lặp ≥3 lần, bỏ giai đoạn khởi động Năng lượng đo bằng tegrastats

III. KẾT QUẢ VÀ KẾT LUẬN

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 1: tách tầng nhanh gấp 24 lần, độ trễ gần như không đổi

Cách triển khai	Phân quyết/giây	Độ trễ p95 (ms)
Một bo	2,6	12,3
A: Tách tầng	62,5	13,1
B: Nhân bản	5,9	21,6
C: Cụm Spark	90,0	23,9

p95: 95% số lượng có phản quyết nhanh hơn mức này

- **Tách tầng: gấp 24 lần**, công xử lý xong 95,8% số luồng trước khi tới Spark.
- **Nhân bản: chỉ gấp 2,3 lần** — bo nào cũng vẫn phải chạy Spark.
- Cụm Spark nhanh nhất nhưng **dao động mạnh** ($\pm 25,6$).

Nên dùng tách tầng: nhanh và ổn định nhất.

2026-09-29
Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark
III. KẾT QUẢ VÀ KẾT LUẬN

Độ lệch giữa các lần lặp: một bo $2,60 \pm 0,41$; tách tầng $62,5 \pm 0,6$; nhân bản $5,9 \pm 1,4$; cụm Spark $90,0 \pm 25,6$ luồng/giây. Độ trễ p95 lệch 3,0 / 2,6 / 19,2 / 17,8 ms. Nhân bản chứng minh lợi ích đến từ việc tách vai trò, không phải từ việc thêm một bo. Cụm Spark còn cần thêm máy master và hợp với mô hình quá lớn cho một bo.

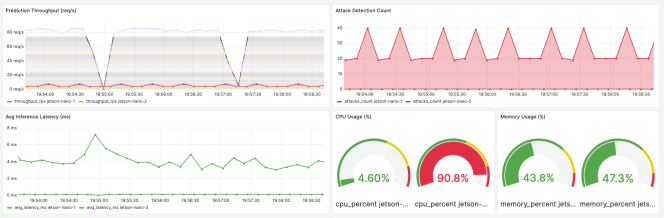
Cách triển khai	Phân quyết/giây	Độ trễ p95 (ms)
Một bo	2,6	12,3
A: Tách tầng	62,5	13,1
B: Nhân bản	5,9	21,6
C: Cụm Spark	90,0	23,9

p95: 95% số lượng có phản quyết nhanh hơn mức này.

Nên dùng tách tầng: nhanh và ổn định nhất.

- **Tách tầng: gấp 24 lần**, công xử lý xong 95,8% số luồng trước khi tới Spark.
- **Nhân bản: chỉ gấp 2,3 lần** — bo nào cũng vẫn phải chạy Spark.
- Cụm Spark nhanh nhất nhưng **dao động mạnh** ($\pm 25,6$).

Kết quả 2: nhìn trực tiếp thấy bo nào đang gánh việc



Lúc tải cao nhất, chế độ tách tầng:

- bo cổng lọc: **4,6% CPU**
- bo phân loại: **90,8% CPU**

Bộ phân loại chiếm gần hết sức tính — nên tách nó ra khỏi đường nhận dữ liệu là đáng.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 2: nhìn trực tiếp thấy bo nào đang gánh việc

Ảnh Grafana chụp khi đang phát lại dữ liệu ở chế độ tách tầng.



Lúc tải cao nhất, chế độ tách tầng:
bo cổng lọc: 4,6% CPU
bo phân loại: 90,8% CPU
Bộ phân loại chiếm gần hết sức tính — nên tách nó ra khỏi đường nhận dữ liệu là đáng.

Kết quả 3: năng lượng mỗi phán quyết giảm từ 2,5 J còn 0,18 J

Cách triển khai J / phán quyết

Một bo 2,49

A: Tách tầng 0,18

B: Nhân bản 2,25

C: Cụm Spark 0,12

- Ở 100 luồng/giây, điện năng *tăng thêm* khi chạy quá nhỏ để đo (<0,15 W).
- Nên con số chủ yếu là **điện nền của bo chia đều cho số phán quyết**.
- Tách tầng và cụm Spark ra nhiều phán quyết hơn 24–35 lần, nên mỗi phán quyết tốn ít hơn hẳn.

Đây là năng lượng ở *mức triển khai*, không phải năng lượng của riêng mô hình.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Năng lượng đo bằng tegrastats: 30 giây nền rồi đến cửa sổ tải; chế độ hai bo cộng cả hai bo.

Kết quả 3: năng lượng mỗi phán quyết giảm từ 2,5 J còn 0,18 J

Cách triển khai	J / phán quyết
Một bo	2,49
A: Tách tầng	0,18
B: Nhân bản	2,25
C: Cụm Spark	0,12

- Ở 100 luồng/giây, điện năng tăng thêm khi chạy quá nhỏ để đo (<0,15W).
- Nên con số chủ yếu là **điện nền của bo chia đều cho số phán quyết**.
- Tách tầng và cụm Spark ra nhiều phán quyết hơn 24–35 lần, nên mỗi phán quyết tốn ít hơn hẳn.

Đây là năng lượng ở mức triển khai, không phải năng lượng của riêng mô hình.

Kết quả 4: cổng lọc cho lọt bao nhiêu tấn công

Ngưỡng cổng (phần v)	Tấn công chuyển tiếp	Báo nhảm	Luồng bỏ qua
0.99	74.2%	1.01%	88.0%
0.995 (đang dùng)	72,3%	0,53%	88,7%
0.999	58.1%	0.10%	91.2%

Đo offline trên tập kiểm tra CICIDS2017.

- Luồng bị cổng bỏ qua là *chốt luôn*: **khoảng 28% luồng tấn công** không tới được bộ phân loại.
- Hạ ngưỡng thì bắt thêm tấn công nhưng Spark phải xét nhiều luồng hơn.
- Người vận hành chọn điểm hợp với sức máy.

Tốc độ gấp 24 lần phải trả bằng recall: cổng là một **sự đánh đổi có thể chỉnh**, không phải bộ lọc miễn phí.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Số liệu từ results/ml_08_anomaly_gate/gate_operating_points.csv. Tỷ lệ bỏ qua ở đây đo offline trên tập kiểm tra; khi chạy thật cổng bỏ qua 95,8% của một mẫu lưu lượng khác, nên hai con số không nhất thiết trùng. Đường cong đầy đủ ở slide dự phòng.

Ngưỡng cổng (phần v)	Tấn công chuyển tiếp	Báo nhảm	Luồng bỏ qua
0.99	74.2%	1.01%	88.0%
0.995 (đang dùng)	72,3%	0,53%	88,7%
0.999	58.1%	0.10%	91.2%

Đo offline trên tập kiểm tra CICIDS2017.

- Luồng bị cổng bỏ qua là *chốt luôn*: **khoảng 28% luồng tấn công** không tới được bộ phân loại.
- Hạ ngưỡng thì bắt thêm tấn công nhưng Spark phải xét nhiều luồng hơn.
- Người vận hành chọn điểm hợp với sức máy.

Tốc độ gấp 24 lần phải trả bằng recall: cổng là một **sự đánh đổi có thể chỉnh**, không phải bộ lọc miễn phí.

Kết quả 5: cái giá của việc dùng Spark để suy luận

Cùng một Random Forest, cùng 29 đặc trưng, cùng một bộ — chỉ đổi bộ máy chạy:

Bộ máy chạy	Luồng/giây	p95 (ms)	RAM (%)	mJ mỗi luồng
PySpark (đang dùng)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

mJ mỗi luồng: năng lượng tăng thêm so với lúc bo nghỉ.

ONNX Runtime nhanh gấp **348 lần** PySpark. Hướng tiếp theo: **huấn luyện bằng Spark, suy luận bằng ONNX/TensorRT**.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 5: cái giá của việc dùng Spark để suy luận

Dùng Spark để suy luận là để cùng một bộ công cụ vừa huấn luyện trên cụm vừa chạy trên thiết bị. Cái giá là JVM: scikit-learn đã nhanh gấp 9 lần, p95 thấp hơn khoảng 11 lần, RAM bằng khoảng nửa. Đo ở batch 10, năng lượng tăng thêm tại mức bão hòa.

Kết quả 5: cái giá của việc dùng Spark để suy luận

Cùng một Random Forest, cùng 29 đặc trưng, cùng một bộ — chỉ đổi bộ máy chạy:

Bộ máy chạy	Luồng/giây	p95 (ms)	RAM (%)	mJ mỗi luồng
PySpark (đang dùng)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

ONNX Runtime nhanh gấp **348 lần** PySpark. Hướng tiếp theo: **huấn luyện bằng Spark, suy luận bằng ONNX/TensorRT**.

Những gì đã chỉ ra

- Tách cổng lọc và bộ phân loại ra hai bo: **2,6** → **62,5 phán quyết/giây**, p95 ≈13 ms.
- Năng lượng mỗi phán quyết: **2,5 J** → **0,18 J**.
- Đổi lại, cổng cho lọt ≈**28%** luồng tấn công ở ngưỡng đang dùng.
- Spark là nút thắt suy luận: ONNX nhanh gấp 348 lần.

Hạn chế

- Hai bo, mạng Wi-Fi, dữ liệu phát lại.
- GPU chưa dùng; chưa thử tấn công né tránh.

Hướng tiếp theo

- Suy luận bằng ONNX/TensorRT.
- Lưu lượng thật; huấn luyện lại khi dữ liệu trôi.

Xin cảm ơn — kính mời câu hỏi

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Nhắc lại ba con số: gấp 24 lần, 13 mili giây, 0,18 J. Và nói thẳng cái giá: khoảng 28% luồng tấn công bị cổng cho qua ở ngưỡng hiện tại.

Kết luận

Những gì đã chỉ ra

- Tách cổng lọc và bộ phân loại ra hai bo: **2,6** → **62,5 phán quyết/giây**, p95 ≈13 ms.
- Năng lượng mỗi phán quyết: **2,5 J** → **0,18 J**.
- Đổi lại, cổng cho lọt ≈**28%** luồng tấn công ở ngưỡng đang dùng.
- Spark là nút thắt suy luận: ONNX nhanh gấp 348 lần.

Hạn chế

- Hai bo, mạng Wi-Fi, dữ liệu phát lại.
- GPU chưa dùng; chưa thử tấn công né tránh.

Hướng tiếp theo

- Suy luận bằng ONNX/TensorRT.
- Lưu lượng thật; huấn luyện lại khi dữ liệu trôi.

Xin cảm ơn — kính mời câu hỏi

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018.

CICIDS2017

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016.

điện toán biên

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009.

cổng lọc

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017.

SHAP

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010.

Spark

[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011.

Kafka

[7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025.

hệ thống gần nhất

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022.

chống rò rỉ

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

III. KẾT QUẢ VÀ KẾT LUẬN

Tài liệu tham khảo chính

Tài liệu tham khảo chính	
[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. <i>Toward generating a new intrusion detection dataset and intrusion traffic characterization</i> . ICISSP 2018.	CICIDS2017
[2] Shi, W. et al. <i>Edge computing: Vision and challenges</i> . IEEE Internet of Things Journal 3(5), 2016.	điện toán biên
[3] Chandola, V., Banerjee, A. & Kumar, V. <i>Anomaly detection: A survey</i> . ACM Computing Surveys 41(3), 2009.	cổng lọc
[4] Lundberg, S. M. & Lee, S.-I. <i>A unified approach to interpreting model predictions</i> . NeurIPS 2017.	SHAP
[5] Zaharia, M. et al. <i>Spark: Cluster computing with working sets</i> . HotCloud 2010.	Spark
[6] Kreps, J., Narkhede, N. & Rao, J. <i>Kafka: a Distributed Messaging System for Log Processing</i> . NetDB 2011.	Kafka
[7] Mohan Kiran Kumar, K. et al. <i>Distributed Intrusion Detection System using Kafka and Spark Streaming</i> . ICVADV 2025.	hệ thống gần nhất
[8] Arp, D. et al. <i>Dos and Don'ts of Machine Learning in Computer Security</i> . USENIX Security 2022.	chống rò rỉ

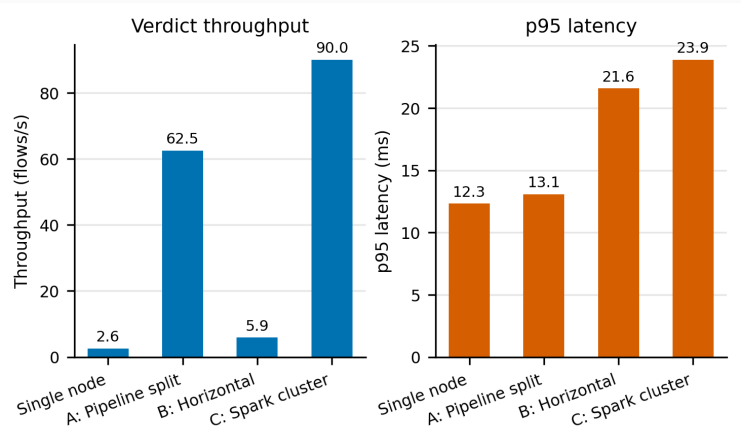
Slide dự phòng

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm
thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Slide dự phòng

Dự phòng: biểu đồ so sánh các cách triển khai

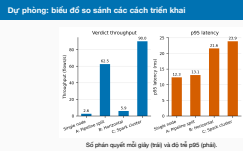


Số phán quyết mỗi giây (trái) và độ trễ p95 (phải).

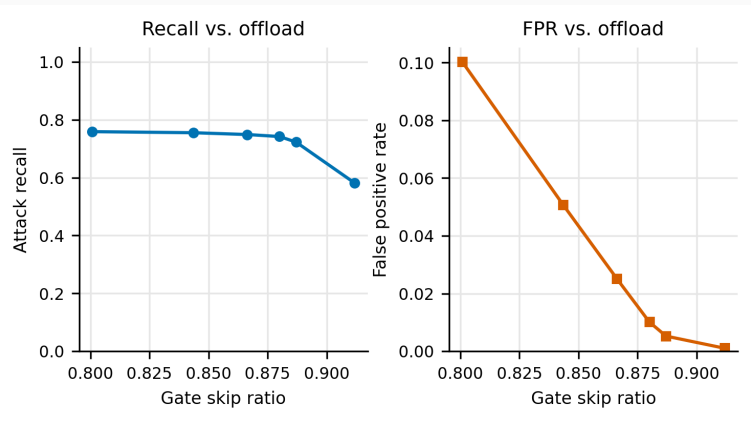
2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Dự phòng: biểu đồ so sánh các cách triển khai



Dự phòng: đường cong vận hành của cổng lọc

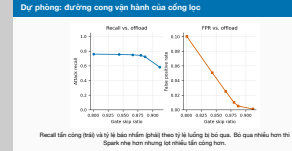


Recall tấn công (trái) và tỷ lệ báo nhầm (phải) theo tỷ lệ luồng bị bỏ qua. Bỏ qua nhiều hơn thì Spark nhẹ hơn nhưng lọt nhiều tấn công hơn.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Dự phòng: đường cong vận hành của cổng lọc



Dự phòng: bảng điều khiển giám sát đầy đủ



Tốc độ từng bộ, số tấn công phát hiện, độ trễ suy luận, CPU/RAM hai bộ, cảnh báo lưu trong PostgreSQL.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Dự phòng: bảng điều khiển giám sát đầy đủ



Dự phòng: quy trình đo

- Máy chủ khởi động các vai trò trên Jetson qua SSH, tạo tải, thu log từng bo.
- Mỗi cấu hình lặp ≥ 3 lần; bỏ giai đoạn khởi động để JVM chưa nóng máy không làm phồng số liệu.
- Truy vấn chỉ số khớp đúng cửa sổ tải, không lấy khoảng thời gian trượt.
- p95 tính theo từng bo trên dữ liệu thô của từng luồng; p95 của cụm lấy bo tệ nhất.
- Độ trễ đầu cuối tính từ lúc gửi tới lúc có phản quyết, đồng hồ đồng bộ NTP, không tính thời gian ghi CSDL.
- Năng lượng: tegrastats, 30 giây nền rồi cửa sổ tải; chế độ hai bo cộng cả hai bo.

2026-09-29

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

└─Dự phòng: quy trình đo

Dự phòng: quy trình đo

- Máy chủ khởi động các vai trò trên Jetson qua SSH, tạo tải, thu log từng bo.
- Mỗi cấu hình lặp ≥ 3 lần; bỏ giai đoạn khởi động để JVM chưa nóng máy không làm phồng số liệu.
- Truy vấn chỉ số khớp đúng cửa sổ tải, không lấy khoảng thời gian trượt.
- p95 tính theo từng bo trên dữ liệu thô của từng luồng; p95 của cụm lấy bo tệ nhất.
- Độ trễ đầu cuối tính từ lúc gửi tới lúc có phản quyết, đồng hồ đồng bộ NTP, không tính thời gian ghi CSDL.
- Năng lượng: tegrastats, 30 giây nền rồi cửa sổ tải; chế độ hai bo cộng cả hai bo.